Heart Disease Classification: Reproducible MLOps Workflow

Course: Machine Learning Operations (MLOps) AIMLCZG523

Assignment: 01

Student: [Add your name and student ID]

Repository: [Add your GitHub repository URL]

Report date: 20 October 2026

1. Executive Summary

This project implements an end-to-end machine-learning workflow for classifying the presence of heart disease from patient measurements. The objective is to demonstrate reproducible data preparation, model selection, experiment tracking, packaging, automated testing, API serving, and operational observability. It is an educational exercise, not a medical product.

The source is the processed Cleveland subset hosted by the UCI Machine Learning Repository. The prepared table contains 303 observations, 13 predictors, and a binary target. A preprocessing pipeline imputes missing values, scales continuous measurements, and one-hot encodes categorical measurements. All learned preprocessing is fitted within cross-validation folds to avoid leakage.

Two candidates are compared: class-balanced logistic regression and a class-balanced random forest. Five-fold stratified cross-validation selects by mean ROC-AUC; logistic regression is selected with mean CV ROC-AUC 0.9025. On one stratified 20% holdout, it achieves accuracy 0.8852, precision 0.8387, recall 0.9286, and ROC-AUC 0.9654. These values describe a small historical benchmark only and do not establish medical utility.

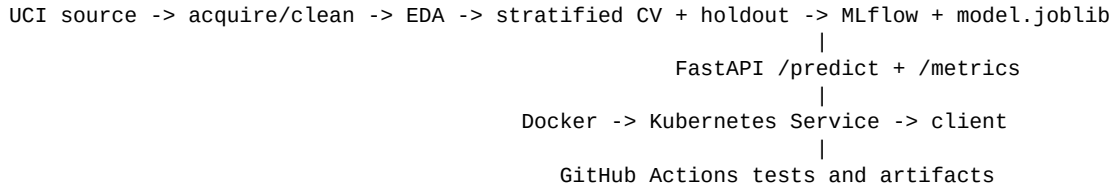
Runs and artifacts are logged with MLflow. The final scikit-learn pipeline is saved as a joblib artifact. FastAPI serves a validated JSON endpoint and exposes Prometheus text counters. Pytest and Ruff run in GitHub Actions, which also trains the model and uploads artifacts. A Dockerfile, Kubernetes Deployment/Service, and Prometheus scrape configuration are included.

2. Problem and System Design

The prediction task is binary classification. The original Cleveland target values are 0, 1, 2, 3, and 4; this workflow defines absence as 0 and presence as any value above 0. Input fields correspond to age, sex, chest-pain type, resting blood pressure, serum cholesterol, fasting blood sugar, resting ECG, maximum heart rate, exercise-induced angina, ST depression, slope, number of major vessels, and thalassemia category.

The system is divided into small components with explicit responsibilities:

- `data.py` downloads, parses, normalizes, and validates the UCI table.
- `modeling.py` builds feature transformations and candidate estimators.
- `train.py` compares candidates, tunes logistic regression, evaluates the holdout, logs MLflow records, and serializes the winner.
- `api.py` loads that artifact and exposes health, prediction, and metrics endpoints.
- `scripts/` contains executable data, EDA, and report workflows.
- `tests/` covers data-target conversion, preprocessing, prediction, request validation, and metrics.



The model is deliberately packaged together with preprocessing. API callers submit original feature values, not scaled or encoded representations. This keeps the serving contract stable and ensures inference applies the same transformations learned during training.

3. Data Acquisition and Preparation

The downloader fetches `processed.cleveland.data` from the UCI archive URL in `src/heart_mlop/data.py`. The raw data has no header, so the script assigns the 14 documented column names. The marker ? is interpreted as missing. Six feature cells are missing in the acquired table; rows are not dropped for those missing predictors. Missing targets are excluded, and the target is mapped to 0/1 before writing `data/processed/heart_disease.csv`.

This repository does not check in a second, manually edited copy of the source. Instead, `python scripts/download_data.py` reproduces the cleaned dataset from the public source. That choice provides clear provenance and avoids silently diverging versions. Network access to the UCI archive is required only for a fresh acquisition; after the file has been downloaded, EDA and training use the local processed CSV.

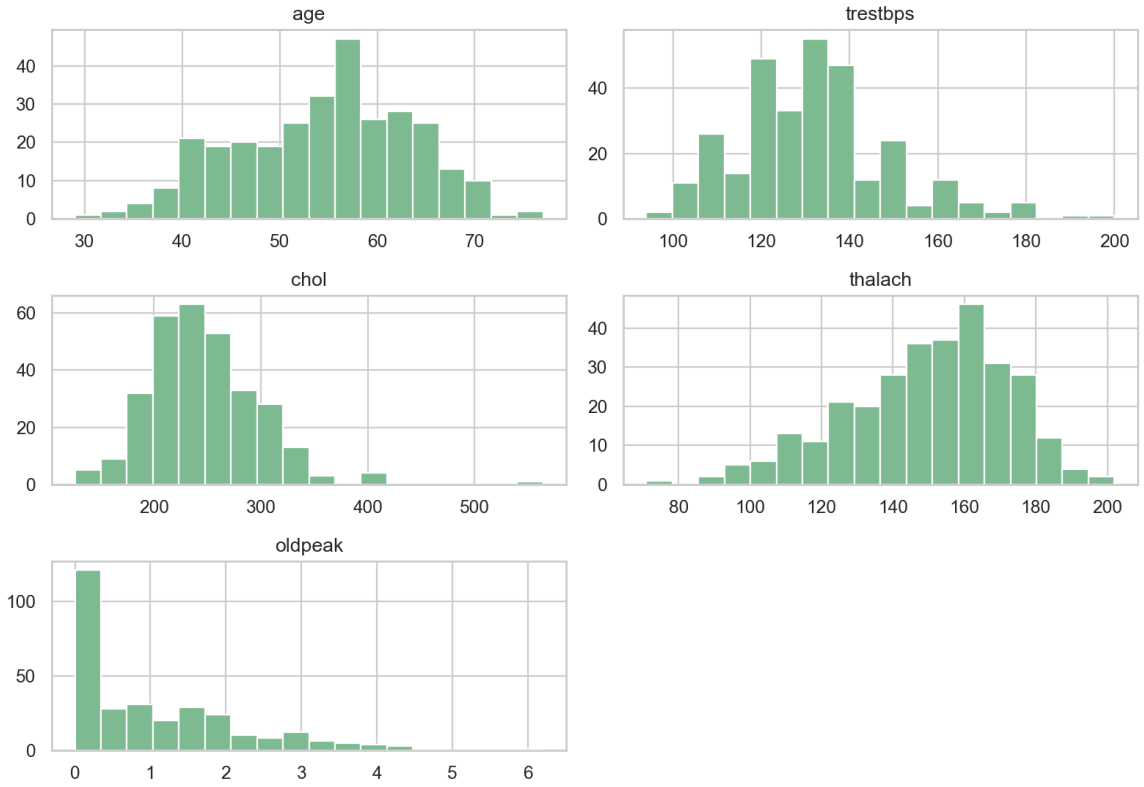
Continuous fields are age, resting blood pressure, cholesterol, maximum heart rate, and ST depression. Category-coded fields are sex, chest-pain type, fasting blood sugar, resting ECG, exercise-induced angina, slope, vessel count, and thalassemia. The code treats the supplied codes as categories rather than assuming numeric distance between codes.

The training/holdout split is stratified, uses 20% of rows for the holdout, and fixes random state 42. Cross-validation uses five shuffled stratified folds, also seeded. Imputers and scalers are inside a scikit-learn `Pipeline` and `ColumnTransformer`; they are refit independently for each training fold. This avoids leaking validation-fold statistics into the fitted preprocessing.

Reproduction commands are in the project README. The acquired file and cleaned table are generated outputs and can be regenerated from the script.

4. Exploratory Data Analysis

The EDA script produces numeric distribution histograms, a target class-balance plot, and a correlation heatmap. Figures are written to `artifacts/eda/`. The histograms help inspect range, skew, and unusual values in the continuous fields. The class plot makes the binary label balance visible. The heatmap is descriptive and uses numeric columns only; it is not treated as evidence of causal relationships.



The Cleveland subset has 303 rows. Its target is moderately imbalanced, which motivates stratified splitting and class-weighted candidate models. Distribution and correlation figures should be interpreted cautiously because the cohort is small, historical, and drawn from a single clinical subset. A high pairwise correlation does not establish that a feature is useful, independent, or causal.

The EDA outputs are generated from the cleaned CSV rather than embedded as hand-edited illustrations. The numeric-distribution figure is included below when this report is built. The companion figure in the artifact directory combines the class-balance chart and correlation heatmap.

```
python scripts/eda.py
```

The command is deterministic with respect to the source table and plotting configuration. If the upstream file changes, regenerated images may change; the exact raw file is preserved under `data/raw/` for the current local run.

5. Feature Engineering and Candidate Models

The numeric branch imputes missing values with the median and standardizes each feature. The categorical branch imputes with the most frequent observed code and applies one-hot encoding with unknown categories ignored at inference. This creates a single feature representation suitable for both candidate classifiers.

The first candidate is class-balanced logistic regression, a compact and interpretable linear baseline. It uses the `liblinear` solver for binary classification and searches regularization strength `C` over 0.01, 0.1, and 1.0 using the same stratified folds and ROC-AUC objective. The final selected estimator includes the chosen setting and preprocessing pipeline.

The second candidate is a class-balanced random forest with 400 trees, minimum leaf size 2, and fixed random seed 42. It provides a nonlinear comparison with no extra tuning search in this baseline assignment. Both candidates use identical preprocessing and train/validation partitions, making the comparison easier to interpret.

The selection metric is mean cross-validated ROC-AUC because it evaluates ranking across thresholds. Accuracy, precision, and recall are reported alongside it to show different operating characteristics. In a real clinical workflow, threshold selection would require stakeholder input, calibrated probabilities, prospective validation, subgroup analysis, and assessment of the costs of false positives and false negatives. None of those clinical deployment claims are made here.

6. Evaluation Results

The table reports mean five-fold CV scores on the training portion, followed by metrics on the single stratified holdout. The holdout was reserved from model selection and is only one estimate on a small dataset.

Candidate	CV accuracy	CV precision	CV recall	CV ROC-AUC	Test accuracy	Test precision	Test recall	Test ROC-AUC
Logistic regression	0.8552	0.8731	0.8099	0.9025	0.8852	0.8387	0.9286	0.9654
Random forest	0.8055	0.8021	0.7735	0.8975	0.8852	0.8182	0.9643	0.9502

Logistic regression is selected because its mean CV ROC-AUC is higher (0.9025 versus 0.8975). On the holdout, the random forest has slightly higher recall while logistic regression has higher precision and ROC-AUC. This tradeoff is why no single metric should be interpreted as a universal measure of quality. The plots `artifacts/plots/logistic_regression/evaluation.png` and `artifacts/plots/random_forest/evaluation.png` show ROC and confusion matrices for the holdout.

The scores are not a reliable estimate of performance on modern populations. The sample is small, feature definitions are historical, and the holdout is a single split. The numbers are included to document the experiment run, not as a clinical claim. A stronger study would use an external dataset and confidence intervals, freeze an evaluation protocol before tuning, and assess calibration and subgroup performance.

7. Experiment Tracking and Reproducibility

MLflow uses the local `mlruns/` file store by default. The experiment name is `uci-heart-disease`; each candidate has a named run. Runs record candidate identity, fold count, split size, seed, feature count, CV metric means and standard deviations, tuning choice where applicable, and holdout accuracy, precision, recall, and ROC-AUC. The evaluation plot and a model artifact are logged for each run.

The selected model is also exported to `artifacts/model.joblib` for straightforward API startup. Its companion `artifacts/model_metadata.json` records the selected candidate, MLflow run ID, ordered feature list, metrics, dataset description, and target rule. The artifact is produced by the training script; it should be regenerated whenever code, dependencies, or data change.

Reproducibility is supported by a requirements file, fixed random seeds, deterministic split definitions, a public data acquisition script, an integrated preprocessing pipeline, model metadata, automated tests, and CI artifact upload. Dependency ranges are bounded rather than fully hash-locked, so this is practical environment reproducibility, not bit-for-bit binary reproducibility. A production release should add a lockfile, source-data checksum, and model/data version identifiers.

Use `mlflow ui --backend-store-uri ./mlruns` to inspect runs locally. For a remote tracking server, set `MLFLOW_TRACKING_URI` before training. Do not put credentials in source control.

8. API Packaging and Validation

The FastAPI application exposes three routes. `GET /health` reports service health and whether the model was loaded. `POST /predict` validates all 13 fields, rejects unrecognized JSON properties, and returns the predicted class, a readable label, confidence in the predicted class, and the probability assigned to the disease-present class. `GET /metrics` exposes request and successful prediction counters in Prometheus text format.

The prediction endpoint logs the class and confidence but does not log the submitted patient record. This reduces unnecessary exposure of health information. There is no authentication, rate limiting, encryption termination, audit retention, or privacy review in this classroom example; it must not be exposed publicly without those controls and appropriate institutional approval.

The tests verify binary target mapping, missing-value imputation through both candidate pipelines, finite predicted probabilities, API response shape, request validation, and the metrics counter. To run the service locally after training:

```
PYTHONPATH=src uvicorn heart_mlop.api:app --host 127.0.0.1 --port 8000
```

The example request is in `examples/patient.json`; interactive OpenAPI documentation is available at `/docs`. The Dockerfile includes the serialized model and source and starts Uvicorn on port 8000. Because Docker was unavailable during preparation, a local image build and container smoke test remain submission tasks.

9. CI/CD, Deployment, and Monitoring

The GitHub Actions workflow runs on pushes and pull requests. It installs requirements on Python 3.11, checks Ruff, fetches and cleans the public data, generates EDA artifacts, trains both models, runs pytest, and uploads the model, figures, and MLflow store even when a later step fails. A test or lint failure fails the workflow.

The Kubernetes manifest defines a two-replica Deployment and LoadBalancer Service, with liveness/readiness probes and CPU/memory limits. Pod annotations expose the `/metrics` scrape path and port. The supplied Prometheus configuration is a local starting point and must be adapted to the actual network target. No Grafana dashboard is bundled. The two counters demonstrate basic request and successful-prediction monitoring, not latency, error rate, model drift, or clinical outcome monitoring.

The expected local sequence is: train; `docker build`; run the container; check `/health`, `/predict`, and `/metrics`; then build/load the image into Minikube and apply `deployment/kubernetes.yaml`. The exact commands are documented in the README. The Docker CLI and Kubernetes tools were not installed in the build environment. Consequently, no container run, cluster deployment, public endpoint, screenshot, or CI execution is represented as completed evidence.

For submission, capture genuine screenshots from a successful CI run, MLflow, API request, and local Kubernetes or cloud deployment. Record a short video of the same workflow. Add the repository URL and access instructions only after publishing the project and verifying the endpoint yourself.

10. Limitations and Reproduction Checklist

This project is a compact teaching implementation. The source cohort is small and old; the model is not clinically validated. The API has no authentication, authorization, encryption policy, request throttling, or protected metrics endpoint. Model monitoring is limited to process-local counters, which reset on restart and are not shared across replicas. There is no alerting, drift detector, model registry promotion policy, or automated rollback. CI uploads evidence but does not publish an image or deploy to a cloud account.

Before submission, personalize the report and complete these steps:

1. Add your name, student ID, course details, and GitHub URL.
2. Create a clean Python 3.11 environment and run data download, EDA, training, lint, and tests.
3. Inspect MLflow runs and verify the stored model and metadata.
4. Build and run the Docker image; send the example request and retain genuine screenshots.
5. Deploy to Minikube or an approved cloud platform; verify the service and `/metrics` from the intended network.
6. Capture the successful CI run and record the required short pipeline video.
7. Update the evidence folder and report to match what you actually ran.

This report intentionally leaves repository identity and deployment evidence for the student to supply. Those are externally dependent facts and must not be fabricated. The local data, metrics, code, and report can all be regenerated using the repository instructions.